

Laboratorio 3

1. Introducción

Un Visitor es un patrón de diseño que define un conjunto de operaciones para cada tipo de nodo en un árbol de sintaxis abstracta (AST), manteniendo separado el procesamiento de la estructura del árbol. Cada nodo del AST implementa un método `accept` que recibe al visitor y llama al método `visit` correspondiente, pasando una referencia a sí mismo. De esta forma, el visitor puede recorrer el AST de manera recursiva: al visitar un nodo binario, por ejemplo, invoca a su vez `accept` sobre los nodos hijos izquierdo y derecho, permitiendo aplicar una lógica específica en cada paso, como evaluar expresiones, generar código o construir una representación gráfica, sin alterar las clases de los nodos.

2. Objetivos

- Comprender la representación de expresiones aritméticas mediante un árbol de sintaxis abstracta (AST).
- Aplicar el patrón Visitor para separar la lógica de procesamiento de la estructura de datos.
- Implementar operaciones de evaluación e impresión de expresiones utilizando visitantes específicos.

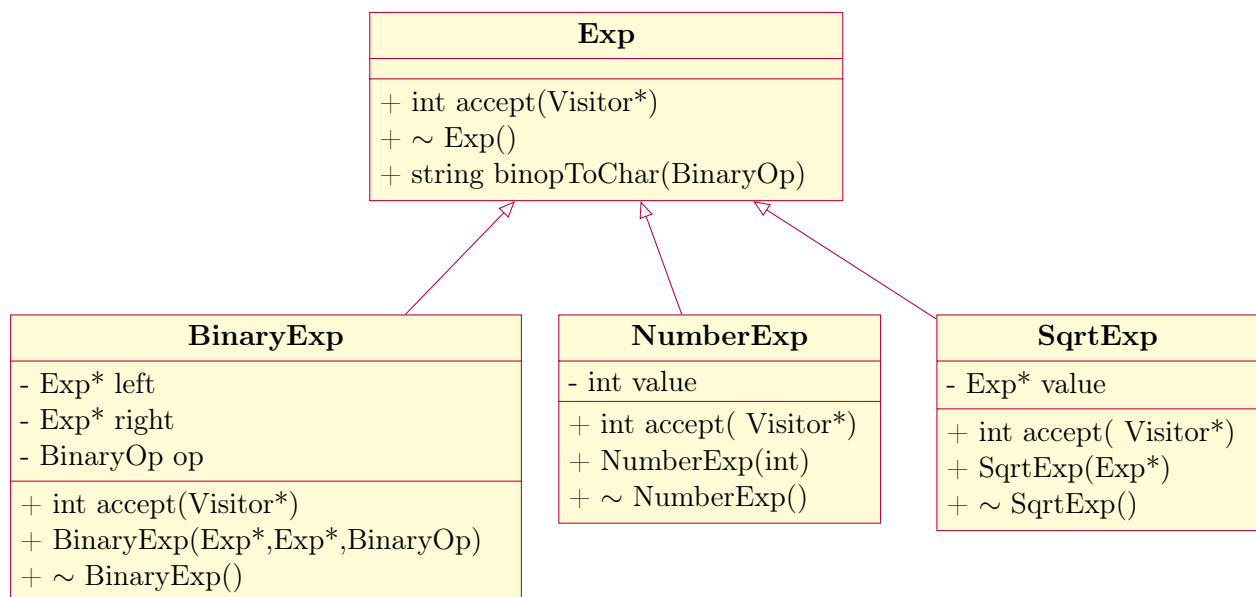
3. Gramática y clases

Se cuenta con un parser que analiza la siguiente gramática de expresiones aritméticas:

$$\begin{aligned} \text{program} &::= \text{expr} \{ (+ \mid -) \text{expr} \}^* \\ \text{expr} &::= \text{term} \{ (* \mid /) \text{term} \}^* \\ \text{term} &::= \text{factor} [* \text{factor}] \\ \text{factor} &::= \text{number} \mid (\text{program}) \mid \text{sqrt}(\text{program}) \end{aligned}$$

Para esta gramática, cada regla se representa mediante una clase dentro de un árbol de sintaxis abstracta (AST). La clase base es `Exp`, que funciona como interfaz común para todas las expresiones. Las expresiones binarias de las reglas `program` y `expr` (operaciones de suma, resta, multiplicación y división) se implementan mediante la clase `BinaryExp`, que contiene punteros a los operandos izquierdo y derecho y un atributo que indica el operador correspondiente. Para la regla `term`, que incluye la potencia (`**`), se reutiliza `BinaryExp` indicando el operador de potencia. Finalmente, las alternativas de la regla `factor` se representan con `NumberExp` para los números y `SqrtExp` para la raíz cuadrada. De esta manera, cada nodo del AST corresponde a un objeto de alguna de estas clases, permitiendo recorrer y procesar la expresión utilizando el patrón Visitor.

- Clase abstracta **Exp**: Representa la clase base de todas las expresiones del AST. Define la interfaz común mediante el método virtual puro `accept(Visitor*)`, que permite aplicar el patrón Visitor a cualquier expresión. Posee un método `binopToChar(BinaryOp)` para convertir operadores binarios a su representación en cadena.
- Clase **BinaryExp**: Representa expresiones binarias que involucran dos operandos y un operador (+, -, *, /, **). Contiene punteros a las expresiones izquierda y derecha (`left` y `right`) y un atributo `op` para el operador. Implementa el método `accept` para que un visitante pueda procesar la expresión..
- Clase **NumberExp**: Representa expresiones numéricas simples, con un atributo `value` que almacena el valor del número. Implementa el método `accept` para permitir el procesamiento por visitantes, y posee constructor y destructor para una correcta gestión de memoria.
- Clase **SqrtExp**: Representa la operación de raíz cuadrada. Contiene un puntero `value` a la expresión cuyo valor se va a calcular, implementa el método `accept` y cuenta con constructor y destructor para una correcta gestión de memoria.

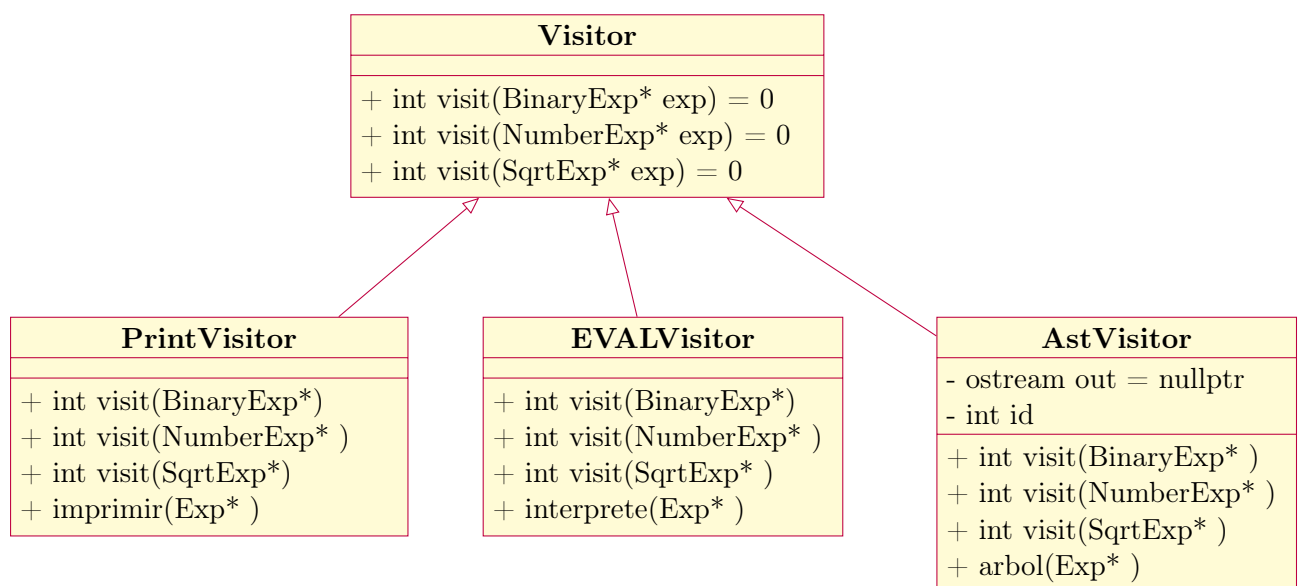


4. Visitor

Se implementó el patrón Visitor para recorrer y procesar el árbol de sintaxis abstracta (AST) generado a partir de la gramática de expresiones aritméticas. Este patrón permite separar las operaciones que se aplican a los elementos de un objeto (en este caso, las expresiones del AST) de la propia estructura de datos, facilitando la extensión de funcionalidades sin modificar las clases existentes. Cada clase de expresión (BinaryExp, NumberExp, SqrtExp) proporciona un método accept que recibe un visitante, delegando la operación correspondiente al método visit adecuado según el tipo de expresión. De esta forma, se mantiene un diseño modular y flexible. Se implementaron los siguientes visitantes:

- EvalVisitor: Calcula y devuelve el valor de la expresión representada en el AST, evaluando recursivamente los nodos y aplicando la operación correspondiente.
- PrintVisitor: Imprime la expresión en notación infija, respetando la precedencia de los operadores y la estructura de los paréntesis, permitiendo obtener una representación legible de la expresión original.
- AstVisitor: Genera una representación visual o textual del árbol completo, mostrando la jerarquía y composición de las expresiones, útil para depuración y análisis de la estructura del AST.

El uso de estos visitantes permite separar la lógica de procesamiento de la estructura de datos, facilitando la incorporación de nuevos tipos de operaciones sin modificar las clases de expresión existentes y fomentando un diseño orientado a objetos, modular y extensible.



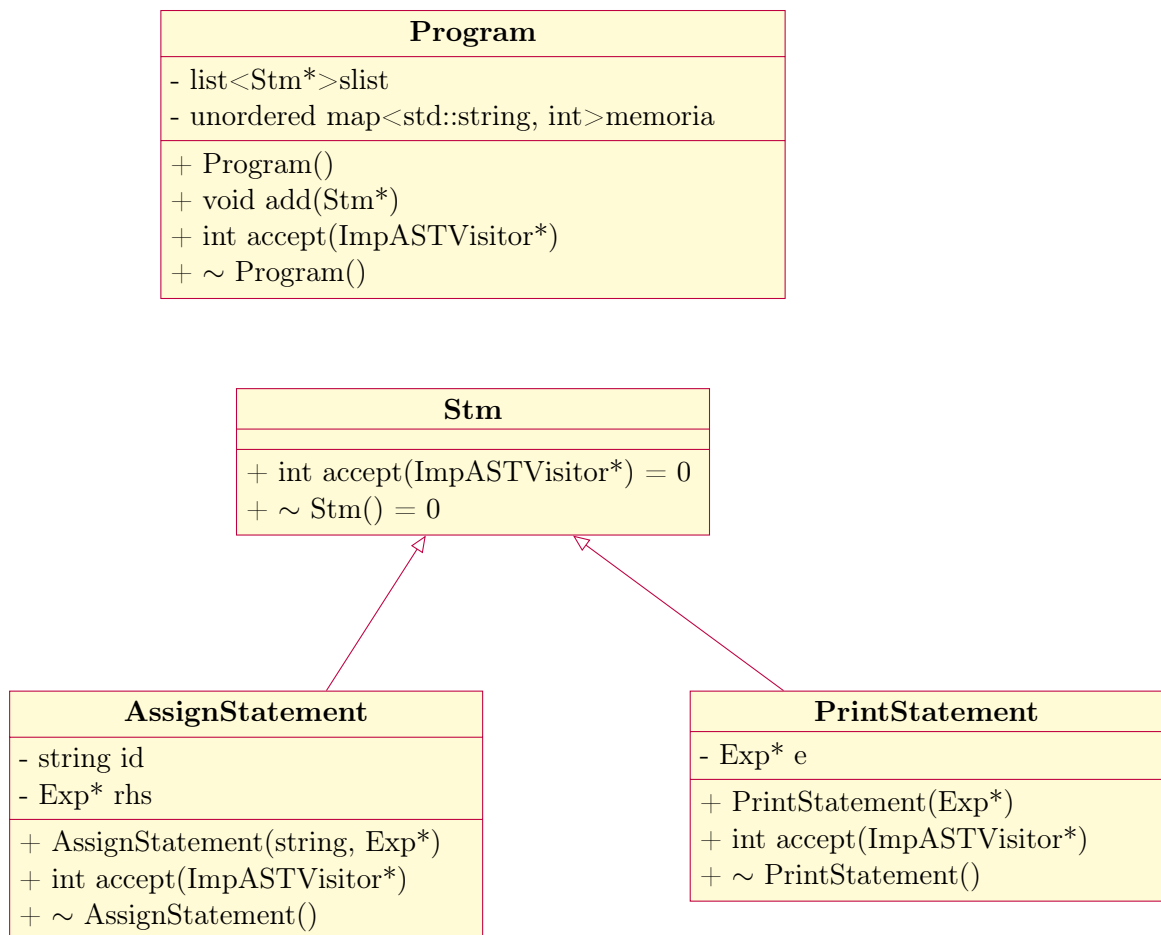
5. Tarea

Implemente un intérprete utilizando el patrón Visitor sobre el árbol de sintaxis abstracta (AST) correspondiente a la siguiente gramática:

$$\begin{aligned} \textit{Program} &::= \textit{StmtList} \\ \textit{StmtList} &::= \textit{Stmt} \text{';' } \textit{Stmt}^* \\ \textit{Stmt} &::= \textit{id} \text{'=' } \textit{CExp} \mid \text{print '(' } \textit{CExp} \text{')'} \\ \textit{CExp} &::= \textit{expr} \{ (+ \mid -) \textit{expr} \}^* \\ \textit{expr} &::= \textit{term} \{ (* \mid /) \textit{term} \}^* \\ \textit{term} &::= \textit{factor} [* \textit{factor}] \\ \textit{factor} &::= \textit{number} \mid (\textit{CExp}) \mid \text{sqrt}(\textit{CExp}) \end{aligned}$$

5.1. Instrucciones:

1. Agregar tokens:
 - a) Defina los tokens que aún faltan para completar la gramática:
 - 1) ASSIGN (=)
 - 2) PRINT (print)
 - 3) SEMICOL (;)
2. Modificar el parser:
 - a) Actualice el parser para que reconozca las reglas de la gramática completa, incluyendo Program, StmtList, Stmt, Exp, Term y Factor.
3. Crear clases AST:
 - a) Implemente las clases necesarias para representar el AST:
 - 1) Program: contiene la lista de sentencias y un diccionario de memoria para las variables.
 - 2) Stm (clase abstracta): base para todas las sentencias.
 - 3) AssignStatement: representa asignaciones de variables.
 - 4) PrintStatement: representa la sentencia print.
 - b) Cada clase debe implementar el método accept para aceptar visitantes y debe gestionar correctamente la memoria.



4. Modificar Visitor:

- Adapte el visitante para procesar los nuevos nodos del AST.
- Implemente al menos los siguientes visitantes:
 - EvalVisitor: evalúa las expresiones y actualiza la memoria de variables.
 - PrintVisitor: imprime las expresiones y sentencias de manera legible.
- Asegúrese de que cada nodo invoque el método visit correspondiente del visitante dentro de accept.